

IM-STYLE · MULTI-AGENT · COLLABORATION

让一群 Coding Agent 真正干活的 IM 协作平台

AgentHub 是一套面向多 Coding Agent 协作的工作台：用户在群聊发目标，Orchestrator 动态生成 DAG,多个 Agent 在真实子对话中执行,主群聊持续呈现计划、进度、产物和最终总结。

8

ARCHITECTURE
LAYERS

4

CODE AGENT BASES

A2A

v0.3 PROTOCOL

1

EVENT-DRIVEN UI

CONTENTS · 目录

六个章节， 讲清 AgentHub 在做什么

从产品定位到分层架构,从执行闭环到 A2A 通信,从技术栈到下一阶段路线——一份可以发给团队、客户或评审人的项目介绍。

01	产品定位与核心体验	P. 03
	Product positioning · 5 core experiences	
02	八层架构模型	P. 04
	Layered architecture · product / orchestrator / protocol / agent / runtime / capability / contract / state	
03	主协作路径	P. 05
	From user prompt to A2A-envelope dispatched task	
04	A2A 通信与产物交接	P. 06
	A2A envelope · workdir · handoff · blackboard	
05	技术栈与项目结构	P. 07
	Tech stack · monorepo · key files	
06	下一阶段路线	P. 08
	Spec Kit contract · AG-UI event convergence	

PART 01 · 产品定位

不是"另一个 Agent 框架", 是多 Agent 协作的 IM 工作台

AgentHub 要解决的是:让一群 Coding Agent 像一支真实团队一样,在用户看得见的群聊里,围绕一个目标真干活,而不是 Orchestrator 一个人扮演所有人。

CORE PROMISE

真执行

子对话里跑真实的 CLI,
不是 LLM 装样子

CORE PROMISE

可见

计划、状态、产物、错误
全过程可追

CORE PROMISE

可控

可停止、可重试、可改派、
可进子会话

CORE PROMISE

可接力

上游产物经 handoff
交给下游,链路完整

五大核心体验

① 私聊 · 全局 Agent 一对一

用户和单个 Agent 直接对话,不进入群聊调度,适合调试、咨询、临时协作。

② 群聊 · 主入口

用户提目标,Orchestrator 决定:简单聊天直接回,复杂任务生成 DAG 与任务看板。

③ 任务子对话 · 真执行

每个任务一个独立子会话,保存 A2A 请求、CLI 流式输出、工具调用和最终结果。

④ 动态任务 DAG · 模型生成

Planner 模型根据上下文产出执行图;系统只校验 schema,不再用关键词或模板路由。

⑤ 产物可见 + handoff 可接力

文件、网页、diff、诊断产物进入消息 metadata 与任务看板;上游产物经 `. agenthub/handoff` 交给下游,下游优先读黑板中明确的 `handoffPath`,而不是臆造相对路径。

关键边界。 AgentHub 不做"固定角色模板系统",也不做"只暴露后端图编排 API 的框架"。它是一个面向多 Coding Agent 的 **IM 形态协作工作台**:WorkBuddy/Kimi 群聊的产品感 × LangGraph/Microsoft Agent Framework 的图与 checkpoint。

把多 Agent 协作切成 8 层, 每层只做一件事

新增任何功能先判断落在哪一层:不要把协议层做成 Agent 类型,不要把工具层做成 runtime,不要让静态代码替 Orchestrator 做任务意图与分工判断。

L8	工作区与状态层 · .agenthub/workdirs · handoff · blackboard · execution_logs · run_events	STATE
L7	协作契约层 · 用户显式 Spec / Contract,只描述范围、产物、验收、路径边界	CONTRACT
L6	能力工具层 · MCP · Skills · Rules · shell · filesystem · browser	CAPABILITY
L5	执行运行时层 · Codex CLI · Claude Code · OpenCode · Gemini CLI · LLM fallback	RUNTIME
L4	Agent 身份层 · code-agent 主路径 · llm 兜底 · role profile · permissions	IDENTITY
L3	通信协议层 · A2A message/send · task · artifact · AG-UI run/step/tool events	PROTOCOL
L2	编排规划层 · Orchestrator · Planner · DAG · TaskScheduler · Synthesizer · HITL	ORCHESTRATION
L1	产品交互层 · 群聊 · 私聊 · 任务子对话 · 任务看板 · 产物卡	PRODUCT

与业内方案对照

业内方案	值得学习	不应照搬
WorkBuddy / Kimi 群聊	IM 体验、状态可见、成员产物可进	不要只做表演式群聊
Claude Code subagents	专家角色、独立上下文、工具权限	不要只局限在单 CLI 内
LangGraph / MS Agent Framework	DAG、状态、handoff、checkpoint	不要把 AgentHub 变成纯后端图框架
A2A / AG-UI / MCP	标准化通信 / 事件 / 工具	不要把它们当 Agent 类型 / runtime

从一条群聊消息， 到一群 Agent 真干活

Orchestrator 是入口,Planner 出图,Scheduler 调度,执行宿主把 A2A envelope 翻译成真实的 Code Agent 进程。每一段都只关心自己那一层。



关键节点说明

1. 群聊入口 · `messages.ts`

用户消息先经 `intentRouter` 判断复杂度;简单聊天由 Orchestrator 直接回复,复杂任务进入动态计划路径。

2. Planner 出 DAG

分工来自 Planner 模型输出;系统代码只校验 schema、权限、Agent 是否存在、依赖是否合法——不再做关键词路由或自动注入。

3. A2A envelope 化

每个任务都被包装为 A2A v0.3 `message/send`,子对话 `user message metadata` 保存这份 envelope,确保子会话是真实上下文。

4. TaskScheduler 按依赖并发

调度器按 DAG 分层执行,前序任务产出写黑板后,后续任务以 `referenceTaskIds` 形式拿到上游信息。

5. LocalA2ATransport 派发

把 envelope 转给本地执行宿主,适配 LLM fallback 或 Code Agent CLI;为每个任务开独立 `workdir` + `sandbox`。

6. 汇报 + Synthesizer 汇总

主群聊发布成员汇报,产物卡可见,Synthesizer 给出最终总结。失败任务也保留部分产物,不显示"无产物"。

A2A 是 协议, 不是 Agent 类型;handoff 是 产物链,不 是路径

A2A 负责 Agent 间 message / task / artifact 语义;AG-UI 负责把运行事件标准化给前端;MCP、Skills、Rules 是 Code Agent 的能力层。AgentHub 把它们各管一层,不互相冒充。

A2A v0.3 envelope 关键字段

```
// AgentHub 内部 A2A message/send envelope(节选)
{
  "type": "message/send",
  "message": {
    "contextId": "ws-main-group-...", // 主群聊 sessionId
    "taskId": "task-...", // workspace_tasks.id
    "referenceTaskIds": ["task-up-1", "task-up-2"], // 上游依赖
    "metadata": {
      "agenthub.dev/a2a/internal": {
        "runId": "run-...",
        "workspaceId": "ws-...",
        "childSessionId": "ses-...",
        "fromAgent": "orchestrator",
        "toAgent": "codex-implementer"
      }
    }
  }
}
```

三层产物 · handoff 是 AgentHub 对 A2A 的扩展

① 消息 metadata · artifacts

子对话和主群聊消息 metadata 中的 artifacts, 作为最轻量的产物索引。

② workspace_tasks.artifacts

任务级产物记录,可被主群聊直接展示成产物卡。

③ blackboard_entries

结构化摘要与 artifact ref,作为下游 Agent 寻找上游信息的事实表。

HANDOFF 规则 · 关键

PART 05 A2A 通信和产物链接
下游 Agent 优先读取黑板中明确给出的 **handoffPath**。filePath / path 只是上游记录,不代表该文件存在于当前 Agent 工作目录;不允许下游 Agent 读取自己目录里臆造的相对路径。 06

Bun + Hono + React,

一套轻量但严肃的栈

Server 走 Hono + Bun.serve,Web 走 React 18 + Vite,持久层用 SQLite + Drizzle,LLM 走 OpenAI / Anthropic 兼容流式协议。Code Agent 不写自家轮子,直接适配 4 个主流 CLI。

技术栈一览

层	技术	用途
运行时	Bun ≥ 1.1.0	单进程 HTTP + WebSocket + TS 执行
Monorepo	Bun workspaces	apps/* 与 packages/* 一仓多包
后端	Hono + Bun.serve + WebSocket	统一一个端口跑 HTTP 与 WS
前端	React 18 + Vite + TypeScript	IM 形态会话界面与任务看板
UI 库	Tailwind + Radix UI + assistant-ui	主题化组件与可访问性
状态	Zustand	客户端轻量 store
数据库	SQLite + Drizzle ORM	本地持久化,零运维
LLM	OpenAI / Anthropic 兼容流式	驱动 Orchestrator / Planner / Synthesizer
Code Agent	Codex CLI / Claude Code / OpenCode / Gemini CLI	主执行基底,真正干活的那一层
Agent 通信	A2A v0.3 message/send	内部任务信封 + 远端 A2A endpoint 预留

关键模块速查

路径	职责
apps/server/src/routes/messages.ts	主入口 · 群聊 / 私聊消息路由
services/orchestrator/orchestrator-engine.ts	调度主循环
services/orchestrator/planner.ts	动态 DAG 生成
services/orchestrator/task-scheduler.ts	按依赖分层执行

下一阶段两条主线: Spec Kit 契约化 + AG-UI 事件收敛

不是把外部框架整套搬进来,只吸收它们成熟的分层思想:Spec Kit 出产物链与质量门,AG-UI 出运行事件到前端的唯一实时协议。

① Contract · 协作契约

用户显式给出的契约,只表达:scope 路径边界 · outputs 产物链与必需产物 · quality 验收标准与质量门 · capabilities 期望的 Skills / MCP / Rules。

不允许表达:triggers / phases / requiredAgents / synthesis / 任何按关键词自动命中的字段。

② AG-UI · 事件唯一收敛

前端所有运行态必须由 run events 驱动:RUN_STARTED · plan.created · task.status · artifact.created · RUN_FINISHED。旧的 task_board:* 只作为兼容输入。

解决:发消息后空白、不知道谁在思考、子对话入口点不开、产物不显示、失败掩盖部分产物。

优先级排序 · 第一阶段

- ✓ **Run lifecycle 统一** · 启动 / 取消 / 停止 / 重试 / 恢复 / stale cleanup / PID 记录,让每条 run 可被完整接管。
- ✓ **事件驱动 UI** · 任务看板、子对话入口、产物卡全部由 run events 驱动,不再依赖前端轮询或独立状态源。
- ✓ **Checkpoint / resume** · 让 DAG、任务状态、child session、blackboard、artifact ref 可恢复——中断后能接着跑。
- ✓ **Skills / MCP 闭环** · Agent 配置 → 执行注入 → 工具调用展示 → 权限管理,完整产品化。
- ✓ **Trace 页面** · 从一次协作 run 反查所有任务、事件、产物、错误和耗时,可读、可分享、可复盘。

判断准则 · 一句话

用户能看见真实执行过程,而不是 Orchestrator 口头声称别人参与;Agent 间任务是 **A2A envelope**,不是隐藏的第二套协议;分工来自 Orchestrator/Planner,不来自静态关键词;Code Agent 是主执行体,LLM 是内部/兜底;任务失败也保留和展示部分产物。

当前阶段

通用主线 + A2A + 本地 workdir

下一阶段

Contract 质量门 + AG-UI 收敛

远期方向

远程 A2A · Docker sandbox · Trace